

Credit Card Fraud Detection

Imbalanced Classification — Machine Learning Project

Kate Pogrebnyakova | Data Scientist

BUSINESS QUESTION

A bank processes 284,807 credit card transactions over 2 days. 492 are fraudulent — just **0.17%** of all transactions. How do we build a model that catches as much fraud as possible without blocking too many legitimate customers?

This is a **precision-recall tradeoff problem**. The cost of missing fraud (bank absorbs the loss) is very different from a false alarm (customer friction). The optimal model minimises total business cost — not statistical accuracy.

WHY ACCURACY IS THE WRONG METRIC

A model that predicts *no fraud* for every transaction achieves **99.83% accuracy** while catching zero fraud cases. This is the core challenge of imbalanced classification.

- **Precision-Recall AUC** (Average Precision) is the correct metric — it focuses entirely on minority class detection performance
- **Precision** = of all fraud alerts fired, what % were real fraud
- **Recall** = of all real fraud cases, what % did we catch
- **ROC-AUC is misleading here** — it looks great even for bad models because the True Negative Rate is inflated when negatives dominate 99.8% of data

THE DATA

Dataset: **Credit Card Fraud Detection** (Kaggle / ULB Machine Learning Group) — 284,807 European cardholder transactions from September 2013.

Metric	Value
Total transactions	284,807
Fraud cases	492 (0.173%)
Legitimate cases	284,315 (99.827%)
Missing values	None
Features	V1–V28 (PCA-transformed, anonymised) + Time + Amount
Target	Class: 0 = legitimate, 1 = fraud
Amount range	€0.00 – €25,691

[Chart not found: c1_class_imbalance]

DATA PREPARATION

Feature scaling

V1–V28 are already PCA-transformed and centred — do **not** scale them again. Only **Time** and **Amount** are scaled with StandardScaler.

Stratified train/test split

With 0.17% fraud rate, a random split could concentrate all fraud in one set. Stratified split preserves the fraud rate in both train (394 cases) and test (98 cases).

SMOTE — Synthetic Minority Over-sampling Technique

SMOTE creates synthetic fraud samples in the training set to balance the classes. **Critical: SMOTE is applied only to the training set.** The test set remains the original imbalanced distribution — the real-world scenario. Applying SMOTE before splitting would leak synthetic samples into evaluation.

Training set after SMOTE: 227,452 fraud | 227,452 legitimate (50/50). Test set unchanged: 98 fraud | 56,863 legitimate (0.17%).

[Chart not found: c2_amount_distribution]

[Chart not found: c4_feature_distributions]

MODELS — THREE APPROACHES TO CLASS IMBALANCE

Three models were trained, each using a different strategy to handle the class imbalance. This demonstrates breadth — all three approaches are valid with different tradeoffs.

Model	Imbalance Strategy	Notes
Logistic Regression	SMOTE on training set	Baseline — linear classifier
Random Forest	class_weight='balanced'	Algorithmic upweighting of minority class
XGBoost	scale_pos_weight = 577	Native XGBoost parameter (neg/pos ratio)

RESULTS

Model	PR-AUC	Recall	Precision	FN (missed)	FP (alerts)
Logistic Regression	0.7249	91.8%	5.8%	8	1,458
Random Forest	0.8653	75.5%	96.1%	24	3
XGBoost	0.8810	83.7%	87.2%	16	12

FN = false negatives (fraud missed — bank absorbs loss). FP = false positives (legitimate transactions blocked — customer friction). XGBoost highlighted: best PR-AUC with balanced FN/FP tradeoff. Test set: 56,961 transactions, 98 fraud cases.

[Chart not found: c6_precision_recall_curves]

[Chart not found: c7_confusion_matrices]

THRESHOLD TUNING — BUSINESS COST OPTIMISATION

Default classification threshold is 0.5. The better question: **what threshold minimises total expected business cost?**

Cost assumptions (illustrative — set by the business):

- **False Negative** (missed fraud): €200 — bank absorbs average fraud loss
- **False Positive** (false alarm): €10 — customer friction, service cost, churn risk

XGBoost probabilities were swept from 0.01 to 0.99 and total expected cost computed at each threshold: **Total Cost = (FN × €200) + (FP × €10)**. The threshold at minimum cost is the operationally correct operating point.

[Chart not found: c9_threshold_optimisation]

This framing mirrors credit risk management — where missing a bad loan costs far more than the review cost of a false flag. Kate's accounting background makes this the natural way to think about the problem.

[Chart not found: c10_feature_importance]

TECHNOLOGY STACK

Area	Tools
Data handling	Python, pandas, numpy
Preprocessing	scikit-learn (StandardScaler, train_test_split)
Imbalance	imbalanced-learn (SMOTE)
Models	scikit-learn (LogisticRegression, RandomForestClassifier)
Gradient boost	XGBoost (XGBClassifier, scale_pos_weight)
Evaluation	precision_recall_curve, average_precision_score, confusion_matrix
Visualisation	matplotlib, seaborn

KEY FINDINGS

- **Accuracy is not the right metric** — 99.83% accuracy from predicting no fraud. PR-AUC is the correct evaluation metric for this problem.
- **XGBoost achieves the highest PR-AUC (0.881)** with a strong balance: catches 83.7% of fraud with 87.2% precision and only 12 false alarms on the test set.
- **Logistic Regression catches more fraud (91.8% recall) but fires 1,458 false alerts** — operationally unworkable. The tradeoff is explicit and measurable.
- **Threshold tuning reduces total business cost** below the statistical default. The optimal threshold is a business decision, not a statistics decision.
- **Three imbalance strategies compared** — SMOTE, class_weight, scale_pos_weight — demonstrating breadth of approach beyond just running a model.